

Proposta de Hackathon

Processamento Inteligente de Documentos (IDP) com AWS AI Services + Agente de IA Generativa

Mentores: Samuel Sá, Jusimar Silva, Wilian Uhlmann

1. Contexto

A empresa **DocuSmart Seguros** é uma seguradora de médio porte que processa centenas de sinistros diariamente. Cada solicitação de sinistro chega como um pacote de documentos em diferentes formatos (PDF, imagens, documentos escaneados) contendo:

- Formulários de abertura de sinistro
- Boletins de ocorrência
- Laudos técnicos e médicos
- Notas fiscais e orçamentos
- Documentos de identidade
- Comprovantes e recibos

Atualmente, o processo é **manual e demorado** (~60 minutos por pacote), exigindo que analistas leiam, classifiquem e digitem informações de cada documento nos sistemas internos. Além disso, quando clientes ligam para o SAC perguntando sobre o status ou conteúdo de seus documentos, os atendentes precisam buscar manualmente nos arquivos.

2. O Problema de Negócio

Desafio	Impacto
Processamento lento	Clientes aguardam dias para ter seus sinistros analisados
Custo operacional alto	Grande equipe dedicada apenas à triagem e digitação
Erros de digitação	Dados incorretos causam retrabalho e atrasos
Dificuldade de busca	Atendentes não conseguem localizar informações rapidamente
Não escala	Em períodos de pico (catástrofes, eventos), o gargalo humano é crítico
Falta de insights	Informações valiosas ficam presas em documentos não estruturados

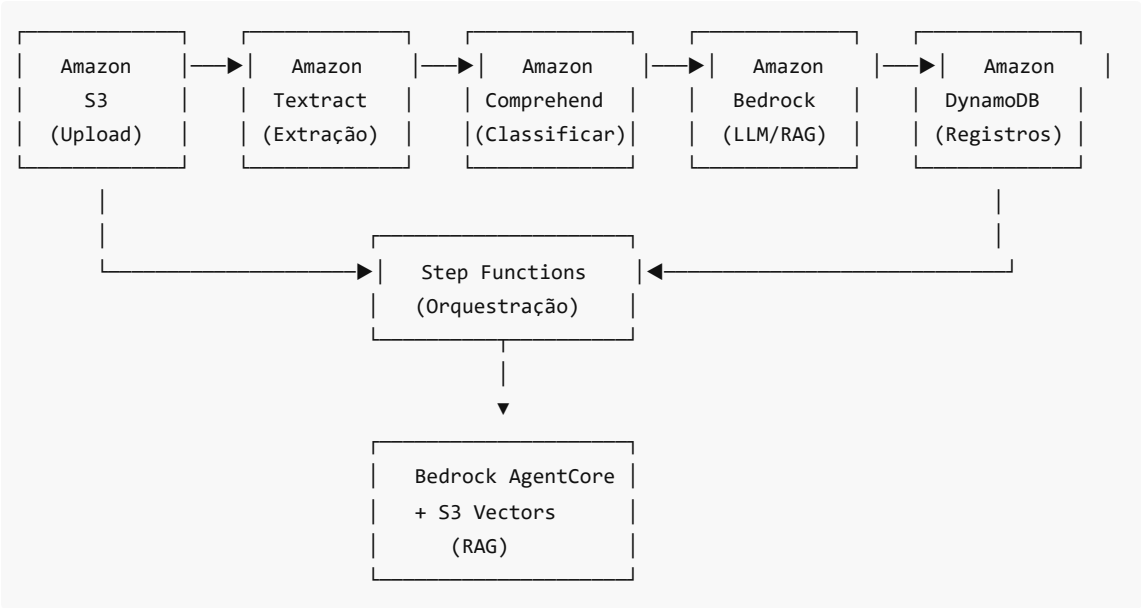
3. O Desafio

Construir uma **solução serverless de IDP** que:

1. **Recebe** pacotes de documentos de sinistros
2. **Classifica** automaticamente cada tipo de documento
3. **Extrai** dados relevantes de forma estruturada
4. **Enriquece** os dados com validações e padronizações
5. **Registra** todas as operações para auditoria e consulta

6. Disponibiliza um **Agente de IA Generativa** para consultas e busca inteligente via RAG

Pipeline de Serviços AWS



4. Componentes da Solução

Serviço	Função
Amazon S3	Armazenamento de documentos originais e processados + Vectors para RAG
Amazon Textract	OCR e extração de texto, tabelas e formulários
Amazon Comprehend	Classificação de documentos e detecção de entidades
Amazon Bedrock	LLMs para normalização, validação e geração de insights
Amazon Bedrock AgentCore	Agente de IA para consultas e RAG
Amazon S3 Vectors	Armazenamento vetorial para busca semântica (RAG)
AWS Lambda	Funções serverless para orquestração
Amazon DynamoDB	Registro de operações e metadados para consulta do agente
AWS Step Functions	Orquestração do workflow de processamento
Amazon API Gateway	APIs REST (GET, PUT, DELETE, POST)

5. Funcionalidades Esperadas

5.1 Pipeline de Processamento (IDP)

- ☐ Upload de documentos via API
- ☐ Classificação automática do tipo de documento

- ☐ Extração de texto, tabelas e key-value pairs
- ☐ Normalização de dados (datas, valores, endereços)
- ☐ Validação de campos obrigatórios
- ☒ **Registro de operações no DynamoDB** para auditoria
- ☐ Geração de JSON padronizado com os dados do sinistro
- ☐ Orquestração completa via **Step Functions**

5.2 Agente de IA Generativa (Diferencial)

- ☐ **Busca Semântica (RAG):** Localizar documentos por conteúdo usando S3 Vectors
- ☐ **Q&A sobre Documentos:** Responder perguntas sobre o conteúdo dos documentos
- ☐ **Consulta de Operações:** Acessar histórico de processamentos via DynamoDB
- ☐ **Resumo de Sinistros:** Gerar resumos executivos dos pacotes de documentos
- ☐ **Assistente de Atendimento:** Auxiliar atendentes a encontrar informações rapidamente
- ☐ **Histórico Contextual:** Manter contexto da conversa para perguntas de follow-up

Exemplos de Interação com o Agente:

 Usuário: "Qual o valor total do orçamento do sinistro 12345?"

 Agente: "O sinistro 12345 possui 2 orçamentos anexados:

- Orçamento A (Oficina X): R\$ 3.500,00
- Orçamento B (Oficina Y): R\$ 3.200,00

O valor médio é R\$ 3.350,00."

 Usuário: "Quais documentos estão faltando nesse sinistro?"

 Agente: "Analisando o sinistro 12345, identifiquei que estão faltando:

- Boletim de Ocorrência
- CNH do segurado

Deseja que eu envie uma notificação ao cliente?"

 Usuário: "Quais operações foram realizadas no documento X hoje?"

 Agente: "O documento X passou pelas seguintes etapas hoje:

- 09:15 - Upload recebido
- 09:16 - Classificado como 'Laudo Técnico'
- 09:18 - Extração concluída (15 campos)
- 09:20 - Validação OK

Status atual: Processado com sucesso."

 Usuário: "Resuma os sinistros de acidentes de trânsito desta semana"

 Agente: "Esta semana foram abertos 47 sinistros de acidentes de trânsito:

- 32 colisões traseiras
- 10 colisões laterais
- 5 outros tipos

Valor total estimado: R\$ 156.000,00"

6. Tecnologias e Pré-requisitos

Conhecimentos Recomendados

- Conceitos básicos de AWS

- Python (boto3)
- Noções de arquitetura serverless
- Conceitos básicos de IA/ML

Requisitos Técnicos

- Conta AWS com acesso ao Amazon Bedrock (região us-east-1)
- Modelos habilitados: Claude 3 Sonnet/Haiku, Titan Embeddings
- AWS CLI configurado
- Python 3.9+

Serviços AWS Utilizados

Serviço	Propósito
Amazon S3	Armazenamento de documentos
Amazon S3 Vectors	Busca vetorial para RAG
Amazon Textract	OCR e Extração
Amazon Comprehend	Classificação e NER
Amazon Bedrock	LLMs e Embeddings
Amazon Bedrock AgentCore	Agente Conversacional
AWS Lambda	Compute Serverless
Amazon DynamoDB	Registro de operações
AWS Step Functions	Orquestração do workflow
Amazon API Gateway	APIs REST

7. Entregáveis

MVP (Mínimo Viável)

1. Pipeline funcional de upload → classificação → extração
2. Endpoints GET, PUT, DELETE, POST funcionais
3. Dados extraídos salvos em formato estruturado (JSON)
4. Registros de operações salvos no DynamoDB
5. Agente básico respondendo perguntas sobre documentos

Completo

1. Tudo do MVP +
2. Interface web para upload e visualização
3. Agente com RAG completo usando S3 Vectors
4. Consulta de histórico de operações via agente
5. Dashboard com métricas de processamento
6. Workflow de revisão humana (A2I) para baixa confiança

8. Critérios de Avaliação

Critério
Funcionalidade completa do pipeline IDP
Qualidade da extração e classificação
Implementação do Agente de IA (RAG)
Arquitetura serverless e boas práticas
Inovação e experiência do usuário
Apresentação e documentação

9. Cronograma Sugerido

Fase	Atividades
Setup	Configurar ambiente, habilitar serviços
IDP Core	Implementar pipeline Textract + Comprehend + Step Functions
API + DynamoDB	Criar endpoints e registro de operações
Bedrock Integration	Adicionar LLM para normalização
Agente RAG	Criar agente com S3 Vectors + Bedrock AgentCore
Integração	Conectar componentes, testar end-to-end
Polish	Documentação, apresentação

10. Recursos e Referências

Workshop Base

- [Intelligent Document Processing with AWS AI Services](#)

Documentação Oficial

- [Amazon Textract Documentation](#)
- [Amazon Comprehend Documentation](#)
- [Amazon Bedrock Documentation](#)
- [Amazon Bedrock Agents](#)
- [AWS Step Functions](#)
- [Amazon DynamoDB](#)

Amazon Bedrock AgentCore

- [AgentCore Samples - GitHub](#)
- [AgentCore Samples - Main Branch](#)
- [Amazon Bedrock AgentCore: Entendendo os desafios de levar agentes de IA para produção](#)

Bibliotecas Úteis

- `amazon-textract-textractor` - Parser de respostas Textract

- `amazon-textract-caller` - Utilitário para chamadas Textract
 - `langchain` - Framework para LLMs
 - `boto3` - SDK AWS para Python
-

11. Dicas para os Participantes

1. **Comece simples:** Primeiro faça o pipeline básico funcionar, depois adicione complexidade
 2. **Use Step Functions:** Facilita a orquestração e visualização do workflow
 3. **Registre tudo no DynamoDB:** O agente precisa dessas informações para responder consultas
 4. **Use os IDP CDK Constructs:** Aceleram muito o desenvolvimento
 5. **Teste com documentos reais:** Use os samples do workshop ou traga seus próprios
 6. **Documente tudo:** Facilita a apresentação final
 7. **Pergunte aos mentores:** Estamos aqui para ajudar!
-

12. Estrutura de Pastas Sugerida

```
projeto-hackathon-idp/
├── README.md
├── infrastructure/
│   ├── template.yaml          # CloudFormation/SAM
│   └── cdk/                   # CDK se preferir
├── lambda/
│   ├── process_document/
│   ├── classify_document/
│   ├── api_handlers/
│   │   ├── get_document.py
│   │   ├── post_document.py
│   │   ├── put_document.py
│   │   └── delete_document.py
│   └── agent_handler/
├── step-functions/
│   └── workflow-definition.json
├── notebooks/
│   ├── 01-textract-extraction.ipynb
│   ├── 02-comprehend-classification.ipynb
│   └── 03-bedrock-agent.ipynb
├── frontend/                  # Opcional
├── samples/
│   └── documents/             # Documentos de teste
└── docs/
    └── api-spec.yaml          # OpenAPI spec
```

Boa sorte e bom hackathon! 🚀